

Asynchronous JavaScript

Synchronous JavaScript

- Is what we have been used to thus far in the class and is how you think programming works as a whole.
- It essentially means that code is executed sequentially from top to bottom
 - Line-by-line
- Imagine there is a long-running task (like reading a file with millions of lines) then the code will have to wait
 - This is called “Blocking I/O” and is very bad for a server
- Imagine if Facebook’s servers processed one request at a time? It could never scale to handle billions of user request

Solution: Asynchronous Programming!

- Asynchronous programming is the concept of when there is a long running task, it is put in the “background” and then the next task is executed
- This is the paradigm that all server code is designed after to handle frequent and large requests

Asynchronous Programming Elucidated

- Also extremely common when dealing with user interfaces
- Think about the concept that UI's will only execute if a specific button is pressed or swipe action is done (waiting for user input)
- We've seen this with jQuery, React, Angular, etc. when we attach actions to specific HTML elements, then execute a function when a user interacts with that element

Familiar Asynchronous Example

```
$( "#dataTable tbody" ).on( "click", "tr", function() {  
    console.log( $( this ).text() );  
});
```

JavaScript is Single Threaded...

- As strange as it sounds, JavaScript is a single threaded language even though there are processes and computations that will be done in the background
- To allow asynchronous programming, Node.js has two things:
 - The Call Stack (sometimes known as just the “Stack”)
 - The Eventloop

The Call Stack

- If you've done recursion, it is very similar
- It stores data about procedures that are currently running in a given program
 - It is actually the “Stack” data structure
- When a function is called it is pushed on the stack
- Once all function calls are pushed, then they are popped (which is invoking them)

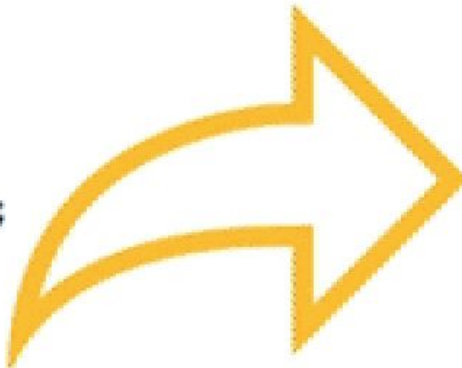
```
function foo(b) {  
    var a = 5;  
    return a * b + 10;  
}
```

```
function bar(x) {  
    var y = 3;  
    return foo(x * y);  
}
```

```
console.log(bar(6));
```



```
1  function foo(b) {  
2    var a = 5;  
3    return a * b + 10;  
4  }  
5  
6  function bar(x) {  
7    var y = 3;  
8    return foo(x * y);  
9  }  
10  
11 console.log(bar(6));
```



Call Stack

console.log(bar(6))

main()

Output:-

```
1  function foo(b) {  
2    var a = 5;  
3    return a * b + 10;  
4  }  
5  
6  → function bar(x) {  
7    var y = 3;  
8    return foo(x * y);  
9  }  
10  
11 console.log(bar(6));
```



Call Stack

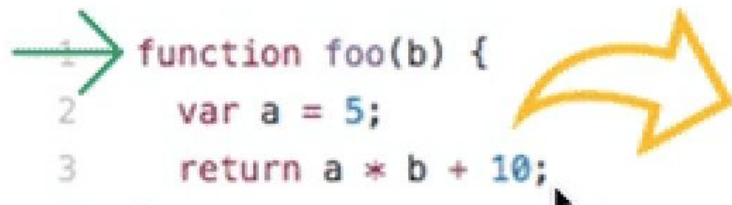
bar(6)

console.log(bar(6))

main()

Output:-

```
1 → function foo(b) {  
2     var a = 5;  
3     return a * b + 10;  
4 }  
5  
6 function bar(x) {  
7     var y = 3;  
8     return foo(x * y);  
9 }  
10  
11 console.log(bar(6));
```



Call Stack

foo(18)

bar(6)

console.log(bar(6))

main()

Output:-

```
1  function foo(b) {  
2    var a = 5;  
3  → return a * b + 10;  
4  }  
5  
6  function bar(x) {  
7    var y = 3;  
8    return foo(x * y);  
9  }  
10  
11 console.log(bar(6));
```

returns 28 and
Pop out

100

Call Stack

foo(18)

bar(6)

console.log(bar(6))

main()

Output:-

```
1  function foo(b) {  
2    var a = 5;  
3    return a * b + 10;  
4  }  
5  
6  function bar(x) {  
7    var y = 3;  
8  → return foo(x * y);  
9  }  
10  
11 console.log(bar(6));
```

100

returns 28 and
Pop out

Call Stack

bar(6)

console.log(bar(6))

main()

Output:-

```
1  function foo(b) {  
2    var a = 5;  
3    return a * b + 10;  
4  }  
5  
6  function bar(x) {  
7    var y = 3;  
8    return foo(x * y);  
9  }  
10  
11 → console.log(bar(6));
```

100

prints 28 on
console and
Pop out

Call Stack

console.log(bar(6))

main()

Output:-

```
1  function foo(b) {  
2    var a = 5;  
3    return a * b + 10;  
4  }  
5  
6  function bar(x) {  
7    var y = 3;  
8    return foo(x * y);  
9  }  
10  
11 console.log(bar(6));
```

script finish,
main pops out,
stack empty

Call Stack

main()

Output:- 28

Callbacks

What is a Callback

- JavaScript functions are considered first-class objects!
- Wut?
 - All that means is that they are treated the same as Numbers, Strings, Objects, etc.
- What that means for a developer is two very important things:
 - Functions can be assigned to variables (we saw this with anonymous functions last week)
 - Functions can be passed as parameters (also saw this with anonymous functions)

Why Use Callbacks

- This is important in JavaScript
- This allows us to pass a **function** to another **function** to be executed asynchronously (aka a later time)

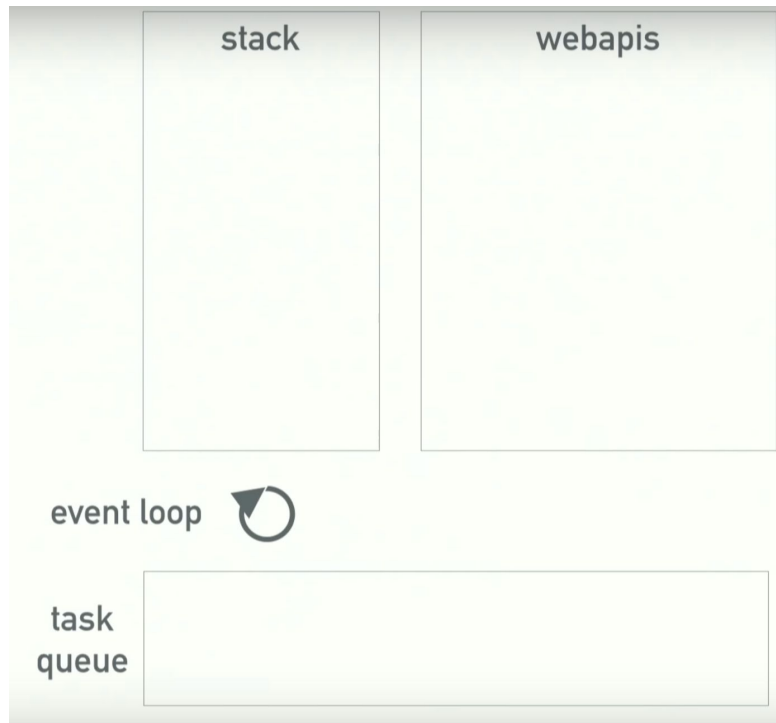
How Callbacks Work

- When normal functions are invoked they are pushed on the call stack like we saw before
- Are these “asynchronous” functions also pushed on the stack?
- **Nah - they're not... initially**
- Callbacks are pushed to some webapis first, then to a thing called the Task Queue

Task Queue

- After a callback function goes to some webapi (don't worry you do not need to know the details of it - for Node it's C++ APIs)
- It then goes to thing called the Task Queue
 - The Task Queue is actually a queue data structure
- Then the Node.js event loop will push the top item of the queue onto the stack

Stack, webapi, task queue, and event loop



P-Rob

<https://youtu.be/8aGhZQkoFbQ?t=52s>

- Watch till minute 16

Thing to Note

- Make sure that tasks relating to an asynchronous call are made inside of the callback function, not outside of it
- Remember, JavaScript will move on and pop things off of the stack before the event loop pushes on async tasks from the task queue

Python vs JavaScript Request Example

- Python request

```
import requests

r = requests.get('http://sohamkamani.com')
print r.text
print "I come after the request"
```


Python vs JavaScript Request Example

- JavaScript Request

```
var request = require('request');

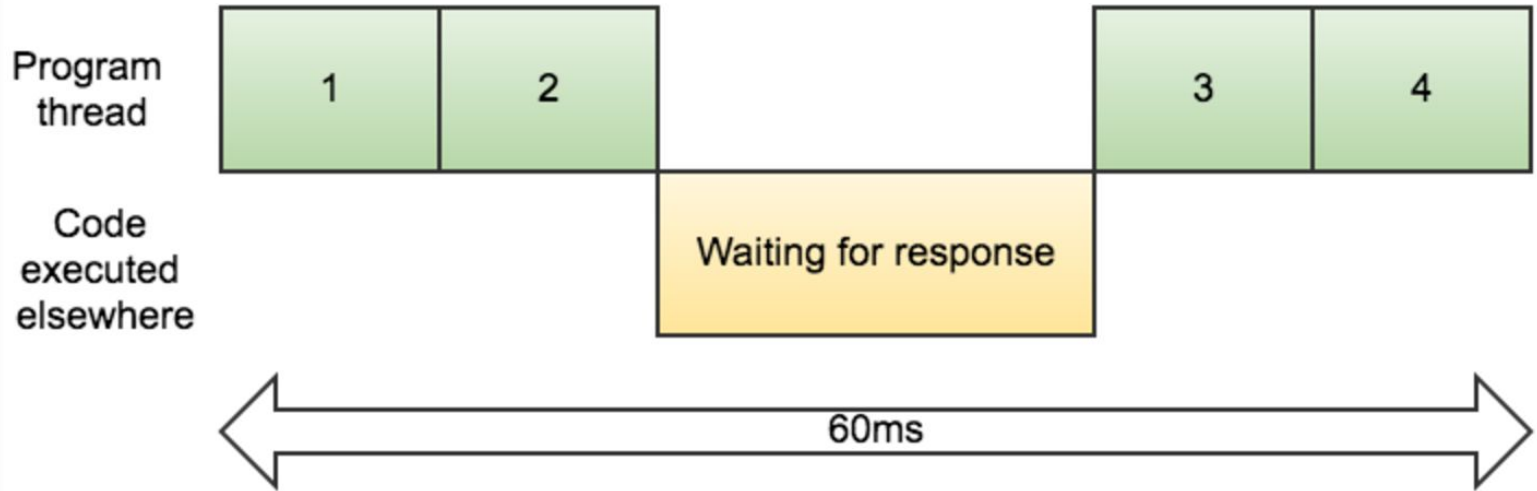
request('http://sohamkamani.com', function (error, response, body) {
  console.log(body);
})

console.log('I come after the request');
```

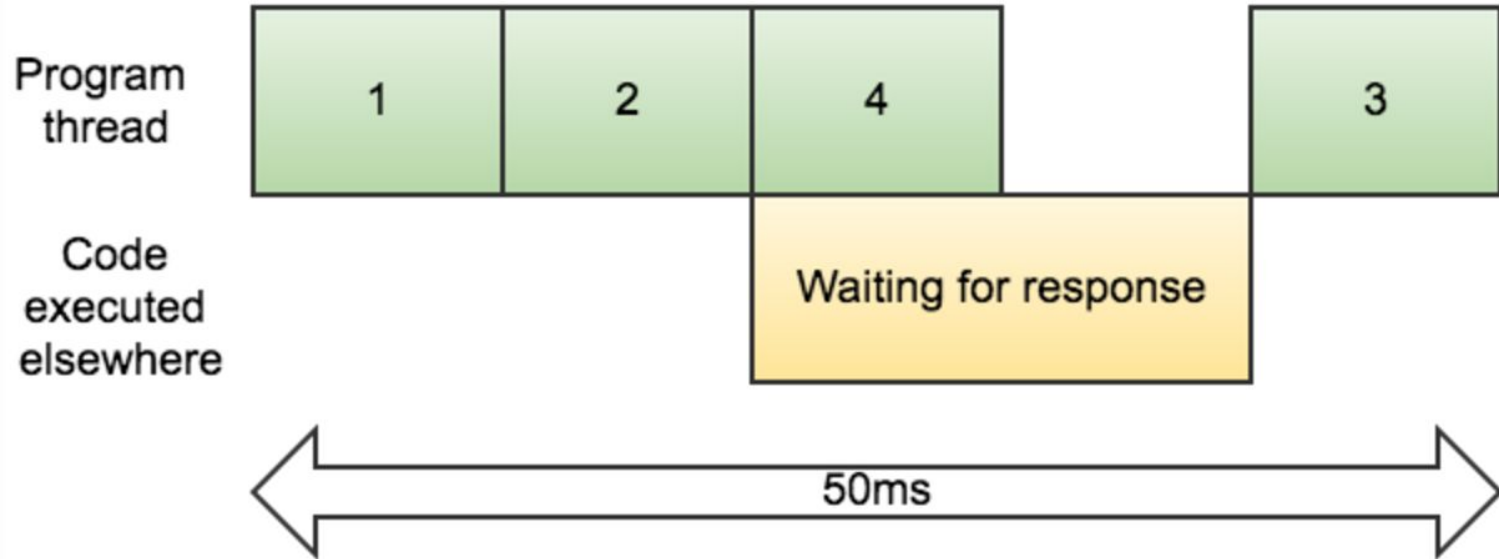
Line-by-Line Comparison

1. `import requests` == `var request = require('request');`
2. `r = requests.get('http://sohamkamani.com')` == `request('http://sohamkamani.com', ... )`
3. `print r.text` == `console.log(body);`
4. `print "I come after the request"` == `console.log('I come after the request');`

In Python...



In JavaScript...



Async Examples

fs.readFile() *wink wink*

- Is the asynchronous version of readFileSync (obvi)
- It has three main parameters
 - 1: The name of the file we are reading from
 - 2: The character set we are using
 - Remember before we needed to use buffer.toString()? Now we just pass into the function the desired character set
 - 3: A callback function that has is given two parameters from the callback function
 - Err - If there is an error in the readFile() function
 - Data - Which is the str returned from reading the file

fs.readdir() *wink wink*

- This function is the asynchronous way to read files in a given directory
- readdir() takes in two parameters:
 - 1: The path to a directory to read from
 - 2: A callback function with two parameters:
 - Err: if there is an error in the readdir() function
 - Data: which is an array of strings with all of the files listed

Path Module *wink wink*

- Install using “npm install path”
- The path module will provide utilities for working with file and directory paths
 - Can make life a tad more easier for the assignment
- The only function that we care about is ‘extname’
- It is invoked using ‘path.extname(filename);’